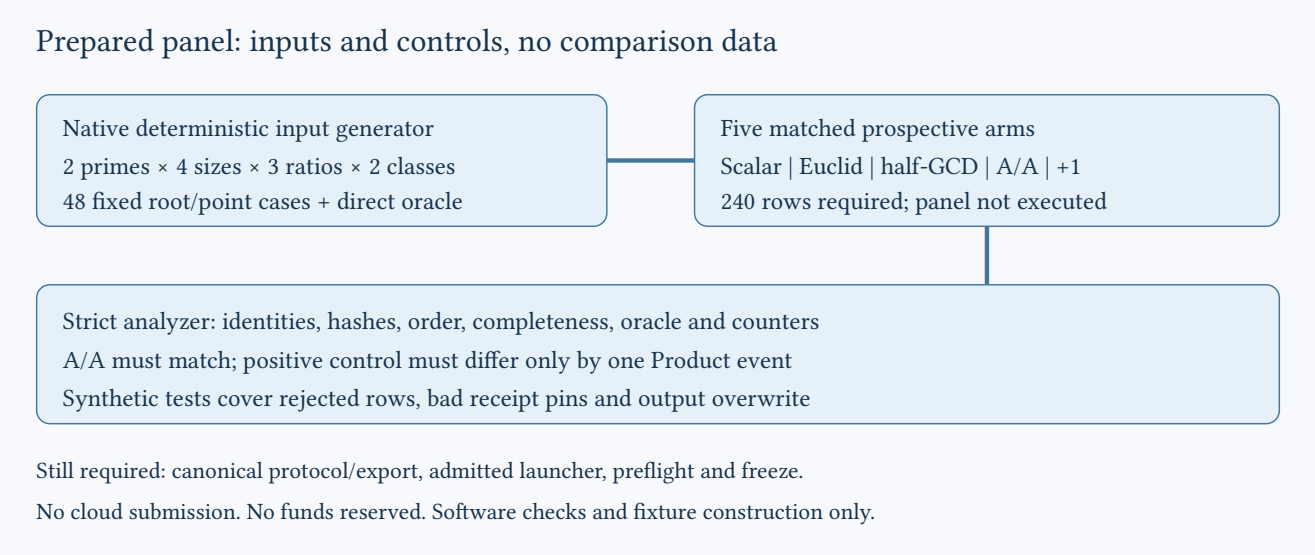
Projected-resultant panel preparation

A deterministic native generator, separate direct-product oracle, planted-product control and strict analyzer now connect to the three preserved accounting backends. No comparison panel was executed.



Software checks

Native p13 controls require identical A/A snapshots and exactly one additional Product event for the positive control, including a common-root fixture. Deterministic input tests verify distinctness, intersections and prime moduli. Source-bound replay checks byte-identical input and metric inventories. Five Python methods test synthetic complete analysis,22 rejected row/control mutations, malformed JSON, six bad external receipt pins and exclusive output creation. Actual commands and outcomes are in validation-01/receipt.json.

Scope and custody

Expected outputs use a separate Cartesian-product oracle outside accounting. Supplied row hashes are identifiers; an admitted external launcher must bind actual files and canonical freeze. The analyzer checks producer receipts but does not authenticate unsigned producers. No weighted cost, timing or fastest-backend conclusion follows from software tests.

Next execution requirements

Canonical protocol/export integration, admitted receipt-producing launcher, preflight, publication and freeze remain required. Proposed one-attempt120s/1CPU/1024MiB/USD1 ceiling; reserved funds0 and remaining budget/capacity unverified. Original4563-case cohort unchanged.

Sources

Preceding accounting source and comparison draft; scalar-resultant direct-product reference; inherited Rust allocator contract. KB unavailable. Producer checks only; no independent review. Two inherited compiler deprecation warnings retained.